Date : 17-03-2026

Relative length units

Relative length units are relative to something else. For example:

- em is relative to the font size of this element, or the font size of the parent element when used for font-size. rem is relative to the font size of the root element.
- vh and vw are relative to the viewport's height and width, respectively.

ems and rems

em - **“my parent element's font-size”** if used for font-size and **"my own font-size"** when used for anything else

rem - **"The root element's font-size"** (rem stands for "root em")

Percentages

a percentage is treated in the same way as a length. The thing with percentages is that they are always set relative to some other value. For example, if you set an element's font-size as a percentage, it will be a percentage of the font-size of the element's parent. If you use a percentage for a width value, it will be a percentage of the width of the parent.

Numbers

Some value types accept unitless numbers; an example is the opacity property, which controls the opacity of an element (how transparent it is). This property accepts a number between 0 (fully transparent) and 1 (fully opaque).

```
.box {
        margin: 40px auto;
        width: 200px;
        background-color: lightblue;
        border: 5px solid darkblue;
        padding: 30px;
        opacity: 0.6;
}
```

Color

Color values can be used in many places in CSS, whether you are specifying the color of text, backgrounds, borders, and lots more. There are many ways to set color in CSS, allowing you to control plenty of exciting properties.

The standard color system available in modern computers supports 24-bit colors, which allows displaying about 16.7 million distinct colors via a combination of different red, green, and blue channels with 256 different values per channel (256 x 256 x 256 = 16,777,216).

Color Keywords

You will see the color keywords (or "named colors") used in many MDN code examples. Because the <named-color> data type contains a very finite number of color values, they are not commonly used on production websites with a sophisticated design language. On the other hand, named colors are used in code examples to clearly tell the user what color is expected so the learner can focus on the content being taught.

Hexadecimal RGB Values

- Hexadecimal numbers use 16 characters from 0-9 and a-f, so the entire range is 0123456789abcdef.
- Each hex color value consists of a hash/pound symbol (#) followed by six hexadecimal characters (#ffc0cb, for example).
- Each **pair** of hexadecimal characters represents one of the channels of an RGB color — red, green, and blue — and allows us to specify any of the 256 available values for each (16 x 16 = 256).

RGB Values:

- To create RGB values directly, the rgb() function takes three parameters representing **red**, **green**, and **blue** channel values of the colors, with an optional fourth value separated by a slash (/) representing opacity, in much the same way as hex values.
- The difference with RGB is that each channel is represented not by two hex digits, but by a decimal number ranging from 0 and 255 or a percentage ranging from 0% and 100% (but not a mixture of the two).

Example:

```
.box {
  padding: 10px;
  margin: 0.5em 0;
  border-radius: 0.5em;
}
.one {
  background-color: rgb(2 121 139);
}
```

```
.two {
  background-color: rgb(197 93 161);
}

.three {
  background-color: rgb(18 138 125);
}
```

RGB Example with Opacity:

```
.one {
  background-color: rgb(2 121 139 / 0.3);
}

.two {
  background-color: rgb(197 93 161 / 0.7);
}

.three {
  background-color: rgb(18 138 125 / 0.9);
}
```

Using hues to Specify a color

If you want to go beyond keywords, hexadecimal, and rgb() for colors, you might want to try using <hue>. Hue is the value type that allows us to tell the difference or similarity between colors like red, orange, yellow, green, blue, etc. The key concept is that you can specify a hue in an <angle> because most of the color models describe hues using a color wheel.

The **<hue>** CSS data type represents the hue angle of a color. It is used in the color functions that accept hue expressed as a single value, specifically hsl(), hwb(), lch(), and oklch() functional notations.

HWB

A great starting point for using hues in CSS is the hwb() function, which specifies an srgb() color. The three parts are:

- Hue: The base shade of the color. This takes a <hue> value between 0 and 360, representing the angles around a color wheel.

- Whiteness: How white is the color? This takes a value from 0% (no whiteness) to 100% (full whiteness).
- Blackness: How black is the color? This takes a value from 0% (no blackness) to 100% (full blackness).

HSL

Similar to the hwb() function is the hsl() function, which also specifies an srgb() color. HSL uses Hue, in addition to Saturation and Lightness:

- Hue: Again, this represents the base shade of the color.
- Saturation: How saturated is the color? This takes a value from 0–100%, where 0 is no color (it will appear as a shade of grey), and 100% is full color saturation.
- Lightness: How light or bright is the color? This takes a value from 0–100%, where 0 is no light (it will appear completely black) and 100% is full light (it will appear completely white).

```
.one {
        background-color: hsl(188 97% 28%);
 }
```

Images:

The <image> value type is used wherever an image is a valid value. This can be an actual image file pointed to via a url() function, or a gradient.

```
.image {
        background-image:url("https://mdn.github.io/shared-assets/images/examples/big-star.png");
}
```

Position:

The <position> value type represents a set of 2D coordinates, used to position an item such as a background image (via background-position). It can take keywords such as top, left, bottom, right, and center to align items with specific bounds of a 2D box, and lengths, which represent offsets from the top and left-hand edges of the box.

Example:

```
.box {
  height: 200px;
  width: 400px;
```

```
  background-image:url("https://mdn.github.io/shared-assets/images/examples/big-star.png");

  background-repeat: no-repeat;

  background-position: right 60px;

  margin: 20px auto;

  border-radius: 0.5em;

  border: 5px solid rebeccapurple;

}
```

Strings and Identifiers:

Throughout the examples above, we've seen places where keywords are used as a value (for example <color> keywords like red, black, rebeccapurple, and goldenrod). These keywords are more accurately described as *identifiers*, a special value that CSS understands. As such they are not quoted — they are not treated as strings.

```
.box {

  width: 400px;

  padding: 1em;

  border-radius: 0.5em;

  border: 5px solid rebeccapurple;

  background-color: lightblue;

}


.box::after {

  content: "This is a string. I know because it is quoted in the CSS.";

}
```

Functions:

In programming, a function is a piece of code that does a specific task. Functions are useful because you can write code once then reuse it many times instead of writing the same logic over and over. Most programming languages not only support functions but also come with convenient built-in functions for common tasks so you don't have to write them yourself from scratch.

we've already seen CSS functions in the Color section above, such as rgb() and hsl().

Aside from applying colors, you can use functions in CSS to do a lot of other things. For example, Transform functions are a common way to move, rotate, and scale elements on a page. You might see translate() for moving something horizontally or vertically, rotate() to rotate something, or scale() to make something bigger or smaller.

Math Functions

When you are creating styles for a project, you will probably start off with numbers like 300px for lengths or 200ms for durations. If you want to have these values change based on other values, you will need to do some math. You could calculate the percentage of a value or add a number to another number, then update your CSS with the result.

CSS has support for Math functions, which allow us to perform calculations in CSS instead of relying on static values or doing the math in JavaScript. One of the most common math functions is calc(), which lets you do operations like addition, subtraction, multiplication, and division.

Example:

```
.wrapper {
  width: 400px;
}
.box {
  padding: 1em;
  border-radius: 0.5em;
  border: 5px solid rebeccapurple;
  background-color: lightblue;
  width: calc(20% + 100px);
}
```

There are many other math functions that you can use in CSS, such as min(), max(), and clamp(); respectively these let you pick the smallest, largest, or middle value from a set of values.

Sizing

The natural or intrinsic size of things

HTML Elements have a natural size, set before they are affected by any CSS. A straightforward example is an image. An image file contains sizing information, described as its **intrinsic size**. This size is determined by the image *itself*, not by any formatting we happen to apply.

If you place an image on a page and do not change its height or width, either by using <img> attributes or CSS, it will be displayed using that intrinsic size.

Setting a specific size:

We can, of course, give elements in our design a specific size. When a size is given to an element (the content of which then needs to fit into that size) we refer to it as an **extrinsic size**.

Using Percentages:

- percentages act like length units
- they can often be used interchangeably with lengths
- When using a percentage, you need to be aware of what it is a percentage *of*. In the case of a box inside another container, if you give the child box a percentage width, it will be a percentage of the width of the parent container.

Percentage margins and padding

If you set margins and padding as a percentage, you may notice some strange behavior.

When you use margin and padding set in percentages, the value is calculated from the **inline size** of the containing block — therefore, the width when working in a horizontal language.

Min- and max- sizes:

we can ask CSS to give an element a minimum or a maximum size. If you have a box that might contain a variable amount of content, and you always want it to be *at least* a certain height, you could set the min-height property on it

Max-width on images:

A common use of max-width is to cause images to scale down if there is not enough space to display them at their intrinsic width, while making sure they don't become larger than that width.

```
<div class="wrapper">
 <div class="box">
  <img
   alt="star"
   class="width"
   src="https://mdn.github.io/shared-assets/images/examples/big-star.png" />
 </div>
 <div class="box">
  <img
   alt="star"
   class="max"
```

```
    src="https://mdn.github.io/shared-assets/images/examples/big-star.png" />
  </div>
  <div class="mini-box">
    <img
      alt="star"
      class="max"
      src="https://mdn.github.io/shared-assets/images/examples/big-star.png" />
  </div>
</div>

.box {
  width: 200px;
}
.mini-box {
  width: 30px;
}
.width {
  width: 100%;
}
.max {
  max-width: 100%;
}
```

Viewport Units:

The viewport — which is the visible area of your page in the browser you are using to view a site also has a size

- vw unit for viewport width
- vh for viewport height

1vh is equal to 1% of the viewport height, and 1vw is equal to 1% of the viewport width

Logical properties and values:

```
.horizontal {
        writing-mode: horizontal-tb;
}
.vertical {
        writing-mode: vertical-rl;
}
```

Logical margin, border and padding Properties:

The margin-top property is mapped to margin-block-start — this will always refer to the margin at the start of the block dimension.

The padding-left property maps to padding-inline-start, the padding that is applied to the start of the inline direction. This will be where sentences start in that writing mode. The border-bottom property maps to border-block-end, which is the border at the end of the block dimension.

```
.logical {
  margin-block-start: 20px;
  padding-inline-end: 2em;
  padding-block-start: 2px;
  border-block-start: 5px solid pink;
  border-inline-end: 10px dotted rebeccapurple;
  border-block-end: 1em double orange;
  border-inline-start: 1px solid black;
}
.physical {
  margin-top: 20px;
  padding-right: 2em;
  padding-top: 2px;
  border-top: 5px solid pink;
  border-right: 10px dotted rebeccapurple;
  border-bottom: 1em double orange;
  border-left: 1px solid black;
}
```

Logical values:

We have so far looked at logical property names. There are also some properties that take physical values of top, right, bottom, and left. These values also have mappings, to logical values — block-start, inline-end, block-end, and inline-start.

you can float an image left to cause text to wrap round the image. You could replace left with inline-start

Backgrounds and borders

Background colors:

The background-color property defines the background color on any element in CSS.

Background images:

The background-image property enables the display of an image in the background of an element.

Controlling Background-repeat:

The background-repeat property is used to control the tiling behavior of images. The available values are:

- no-repeat — stop the background from repeating altogether.
- repeat-x — repeat horizontally.
- repeat-y — repeat vertically.
- repeat — the default; repeat in both directions.
- space — repeat as many times as possible, adding space between the images if there is extra space available.
- round — similar to space, but stretches the images to fill any extra space

Sizing the background image:

the background-size property to size the image to fit inside the background.

background-size can take two <length> or <percentage> values to specify the image's size on the horizontal and vertical directions, or the following keywords:

- cover — the browser will make the image just large enough so that it completely covers the box area while still retaining its aspect ratio. In this case, part of the image is likely to end up outside the box.
- contain — the browser will make the image the right size to fit inside the box. In this case, you may end up with gaps on either side or on the top and bottom of the image, if the aspect ratio of the image is different from that of the box.

Positioning the background image:

Default background-position value is (0,0).

The background-position property allows you to choose the position in which the background image appears on the box it is applied to. This uses a coordinate system in which the top-left-hand corner of the box is (0,0), and the box is positioned along the horizontal (x) and vertical (y) axes.

The most common background-position values take two individual values — a horizontal value followed by a vertical value. You can use keywords such as top and right.

background-position: top center;

We can also use the lengths and percentages.

background-position: 20px 10%;

We can also mix keyword values with lengths or percentages, in which case the first value refers to the horizontal position and the second refers to the vertical position.

background-position: 20px top;

We can also use a 4-value syntax to indicate a distance from certain edges of the box. Each pair of values represents the edge of box to offset from, and the size of the offset from that edge.

background-position: top 20px right 10px;

Gradient backgrounds

A gradient — when used for a background — acts just like an image and is also set by using the background-image property.

```
.a {
  background-image: linear-gradient(
    105deg,
    rgb(0 249 255 / 100%) 39%,
    rgb(51 56 57 / 100%) 96%
  );
}


.b {
  background-image: radial-gradient(
    circle,
```

```
  rgb(0 249 255 / 100%) 39%,
  rgb(51 56 57 / 100%) 96%
 );
 background-size: 100px 50px;
}
```

Multiple background images

It is also possible to specify multiple background images in a single declaration. You do this by specifying multiple background-image values separated by commas.

The other background-* properties can also have comma-separated values in the same way as background-image:

```
 url("image1.png"), url("image2.png"), url("image3.png"), url("image4.png");
background-repeat: no-repeat, repeat-x, repeat;
background-position:
 10px 20px,
 top right;
```

Background attachment

Another option we have available for backgrounds is specifying how they scroll when the content scrolls. This is controlled using the background-attachment property, which can take the following values:

- scroll: causes the element's background to scroll when the page is scrolled. If the element content is scrolled, the background does not move. In effect, the background is fixed to the same position on the page, so it scrolls as the page scrolls.
- fixed: causes an element's background to be fixed to the viewport so that it doesn't scroll when the page or element content is scrolled. It will always remain in the same position on the screen.
- local: fixes the background to the element it is set on, so when you scroll the element, the background scrolls with it.

Using the background shorthand properties:

- There are a few rules that need to be followed when writing background image shorthand values, for example:
- A `background-color` may only be specified after the final comma.

- The value of `background-size` may only be included immediately after `background-position`, separated with the `/` character, like this: `center/80%`.

```
.box {
  width: 500px;
  height: 300px;
  padding: 0.5em;
  background:
    linear-gradient(
      105deg,
      rgb(255 255 255 / 20%) 39%,
      rgb(51 56 57 / 100%) 96%
    )
    center center / 400px 200px no-repeat,
  url("https://mdn.github.io/shared-assets/images/examples/big-star.png")
    center no-repeat,
  rebeccapurple;
}
```

Accessibility considerations with background:

When placing text on top of a background image or color, you should take care that you have enough contrast for the text to be legible for your visitors. When specifying an image with text content on top of it, you should also specify a background-color that will allow the text to be legible if the image does not load.

Screen readers cannot parse background images; therefore, they should be purely decorative. Any important content should be part of the HTML page and not contained in a background.

Borders:

we add borders to an element with CSS we use the border shorthand property to set the color, width, and style of the border on all four sides of a box in one declaration:

```
.box {
  border: 1px solid black;
}
```

The individual properties include the border-width, border-style, and border-color shorthand properties:

```
.box {
  border-width: 1px;
  border-style: solid;
  border-color: black;
}
```

Rounded corners:

We can add rounded corners to a box using the border-radius property and associated longhands that relate to each corner of the box. Two lengths or percentages may be used as a value, the first value defining the horizontal radius, and the second the vertical radius. In a lot of cases, you will only pass in one value, which will be used for both.

For example, to give all four corners of a box a 10px radius:

```
.box {
  border-radius: 10px;
}
```

Or to make the top right corner have a horizontal radius of 1em, and a vertical radius of 10%:

```
.box {
  border-top-right-radius: 1em 10%;
}
```

Overflow

Overflow happens when there is too much content to fit in a box.

CSS tries to avoid “data loss”:

Wherever possible, CSS does not hide content. This would cause data loss. The problem with data loss is that you, or visitors to your website, may not notice. If the submit button on a form disappears and no one can complete the form, this could be a big problem! Instead, CSS overflows in visible ways. You are more likely to see there is a problem. At worst, a site visitor will let you know that content is overlapping.

If you restrict a box with a width or a height, CSS trusts you to know what you are doing. CSS assumes that you are managing the potential for overflow. In general, restricting the block dimension is problematic when the box contains text. There may be more text than you expected when designing the site, or the text may be larger (for example, if the user has increased their font size).

The overflow property:

The overflow property allows you to specify how the browser should handle overflowing content. The default value of the <overflow> value type is visible. With this default setting, one can see content when it overflows.

Hiding overflowing content:

To hide content when it overflows, you can set overflow: hidden. This does exactly what it says: it hides overflow. Beware that this can make some content invisible. You should only do this if hiding content won't cause problems.

Scrolling overflowing content:

When we set **overflow: scroll** this allows the users to scroll and see the overflowing content.

To just scroll on the y axis, you could use the overflow-y property, setting overflow-y: scroll

We can also enable scrolling along the x-axis by using overflow-x, although this is not a recommended way to accommodate long words! If you have a long word in a small box, consider using the word-break or overflow-wrap properties.

We can specify x- and y-axis scrolling using the overflow property, passing two values. If two keywords are specified, the first applies to overflow-x and the second applies to overflow-y. Otherwise, both overflow-x and overflow-y are set to the same value. For example, overflow: scroll hidden would set overflow-x to scroll and overflow-y to hidden.

Only displaying scroll bars when needed:

scrollbars to appear when there is more content than can fit in the box, use overflow: auto

Unwanted overflow in web design:

Modern layout methods manage overflow.

They largely work without assumptions or dependencies for how much content there will be on a web page

When developing a site, always keep overflow in mind. Test designs with large and small amounts of content. Increase and decrease font sizes by at least two increments. Ensure your CSS is robust. Changing overflow values to hide content or to add scrollbars is reserved for a few select use cases (for example, where you intend to have a scrolling box).